



PORTFOLIO • PERSONAL PROJECT

StudyGenius

A smart study assistant powered by a cloud-based generative AI API, turning any course content into structured summaries, personalized quizzes, justified exam predictions, and concept-by-concept progress tracking.

React

Node.js

Express

MongoDB

Tailwind CSS

shadcn/ui

Cloud Generative AI API

JWT

1 Project Vision

StudyGenius was born from a simple observation: studying efficiently takes a lot of time, especially when students have to identify the important concepts themselves, build revision material, and guess what might come up in the exam. StudyGenius automates this work using artificial intelligence.

The user simply uploads their course content (PDF or text), and the application automatically generates a clear, structured summary, quizzes adapted to their level, a well-justified estimate of likely exam questions, and a progress dashboard showing which concepts are already mastered and which need review.

Main goal: turn raw course material into a complete, personalized, and measurable revision journey in just a few seconds.

2 Technology Stack

Frontend

Technology	Role in the project
React (Vite)	Building the user interface with reusable components
Tailwind CSS	Fast, consistent styling without heavy custom CSS
shadcn/ui	Modern, accessible UI components (buttons, cards, modals)
React Router	Navigation between pages (Dashboard, Quiz, Progress, etc.)
React Query	API call management, caching, and automatic refetching
Recharts	Progress-tracking charts by subject and by concept

Backend

Technology	Role in the project
Node.js + Express	REST API server handling all business logic
MongoDB + Mongoose	NoSQL database storing courses, quizzes, summaries, and progress
JWT	Secure user authentication
Multer	Handling course file uploads (PDF, text)
pdf-parse	Automatic text extraction from uploaded PDFs

AI Layer

Component	Role in the project
Cloud Generative AI API	Generates summaries, quizzes, and exam predictions from course content
Structured prompt engineering	Precise instructions sent to the AI to get reliable, well-formatted (JSON) responses

Other Tools

Tool	Role in the project
Cloudinary / local storage	Storing uploaded course files
Socket.IO optional	Real-time feedback while AI content is being generated

3 Overall Architecture

The diagram below shows how the different parts of the application communicate, from the user's browser all the way to the database, passing through the AI service.



4 Folder Structure

Backend

```
studygenius-backend/
src/
  config/
    db.js
    aiClient.js
  models/
    User.js
    Course.js
    Summary.js
    Quiz.js
    ExamPrediction.js
    Progress.js
  controllers/
    authController.js
    courseController.js
    summaryController.js
    quizController.js
    predictionController.js
  services/
    aiService.js
    pdfExtractor.js
    progressService.js
  routes/
    auth.routes.js
    course.routes.js
    quiz.routes.js
    prediction.routes.js
  middlewares/
    authMiddleware.js
    errorHandler.js
  app.js
server.js
.env
```

Frontend

```
studygenius-frontend/
src/
  components/
    ui/
      SummaryCard.jsx
      QuizPlayer.jsx
      PredictionCard.jsx
      ProgressChart.jsx
  pages/
    Dashboard.jsx
    CourseUpload.jsx
    QuizPage.jsx
    PredictionsPage.jsx
    ProgressPage.jsx
  hooks/
    useCourses.js
    useQuiz.js
  services/
    api.js
  store/
    authStore.js
App.jsx
```

5 Data Model

Every piece of information in the application is organized into six main collections in MongoDB:

Collection	Content
User	Name, email, encrypted password, study level
Course	Title, extracted raw content, source file, subject, import date
Summary	Linked course, structured content (sections, key points), generation date
Quiz	Linked course, list of questions, type (MCQ / true-false / open), difficulty level
ExamPrediction	Linked course, predicted questions, justification, estimated probability
Progress	User, course, mastered concepts, overall score, quiz history

6 Core Features

- ### 1. Content Import

The user uploads a PDF file or pastes text directly. The system automatically extracts the textual content and prepares it for analysis.
- ### 2. Structured Summary

The AI generates a hierarchical summary of the course: titles, sub-points, and key definitions, organized clearly for easy revision.
- ### 3. Personalized Quizzes

Multiple-choice questions are automatically generated, adapted to the chosen difficulty level, with instant correction and explanations for the right answer.
- ### 4. Exam Predictions

The application analyzes the course content to identify the concepts most likely to appear on the exam, justifying each prediction (concept frequency, course structure, types of exercises encountered).
- ### 5. Progress Tracking

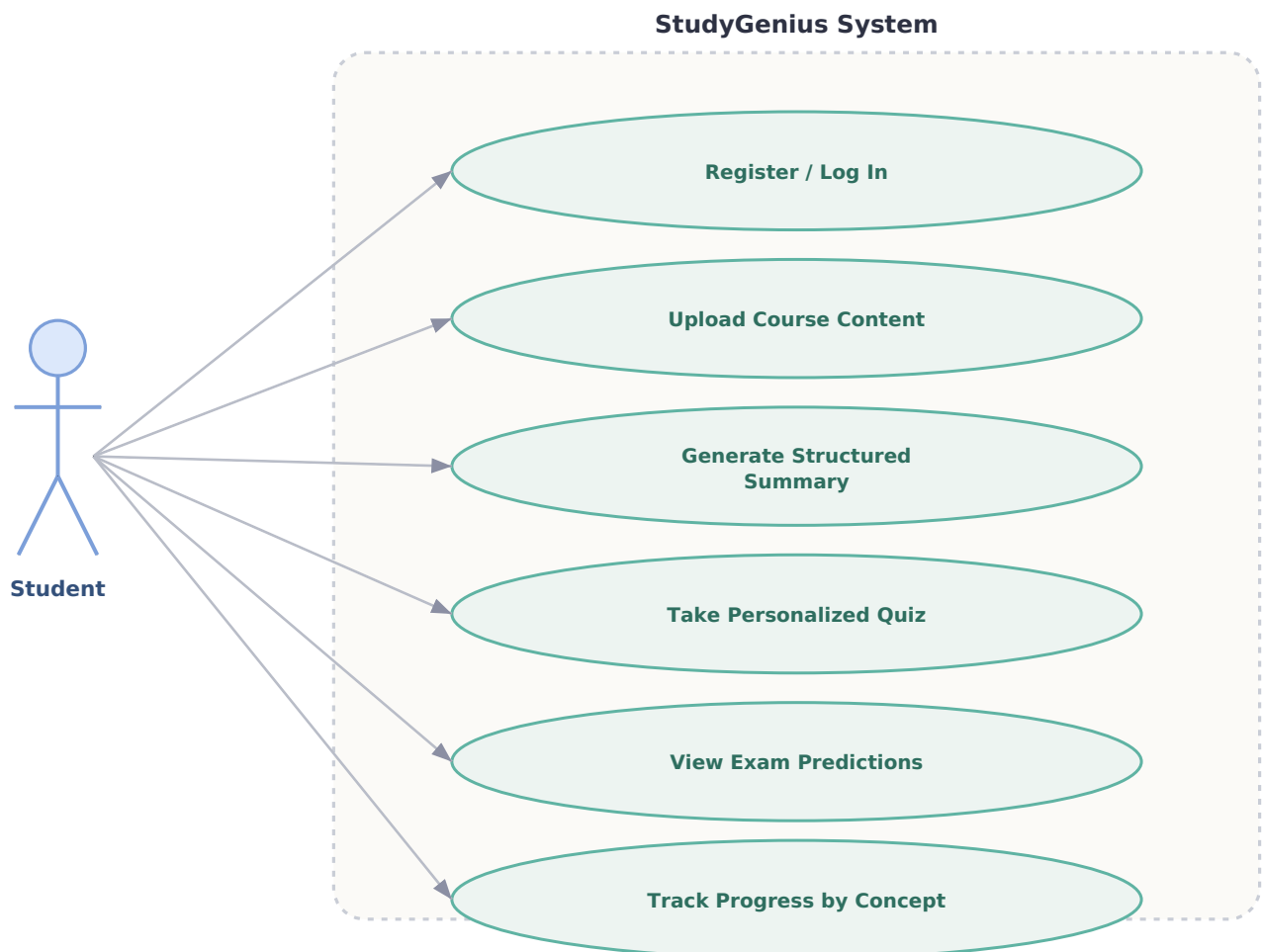
A visual dashboard shows which concepts are already mastered and which need review, along with score evolution over time, subject by subject.

7 Development Plan (Sprints)

- Sprint 1** — Project setup, JWT authentication, PDF upload and text extraction
- Sprint 2** — AI API integration for automatic summary generation
- Sprint 3** — Quiz module: question generation and quiz-taking interface
- Sprint 4** — Exam prediction module with justification
- Sprint 5** — Progress dashboard with charts
- Sprint 6** — UI polish (Tailwind + shadcn/ui), testing, deployment

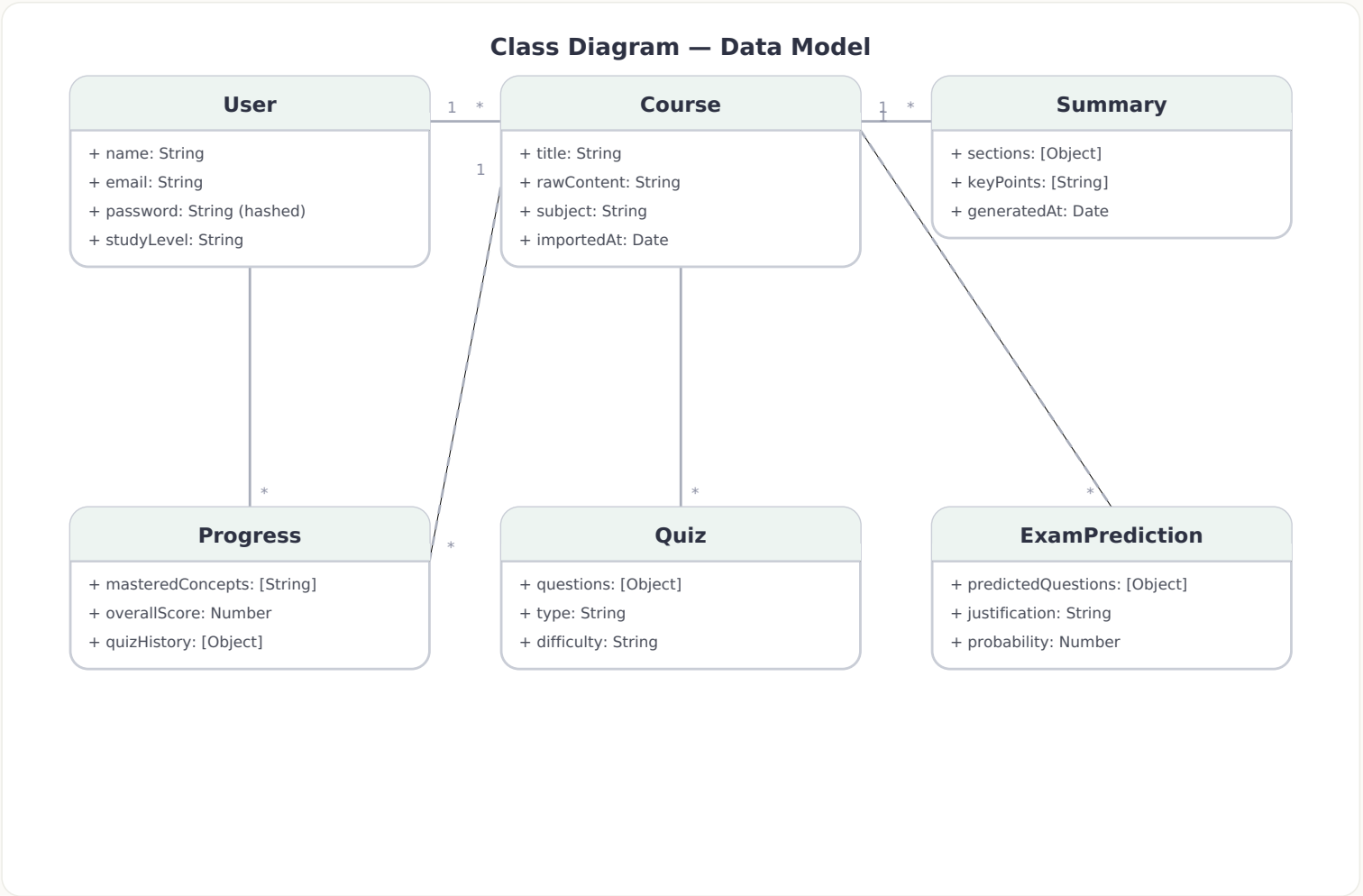
8 Use Case Diagram (General)

The diagram below summarizes, from the student's point of view, every major action available in the application, all grouped inside the StudyGenius system boundary.



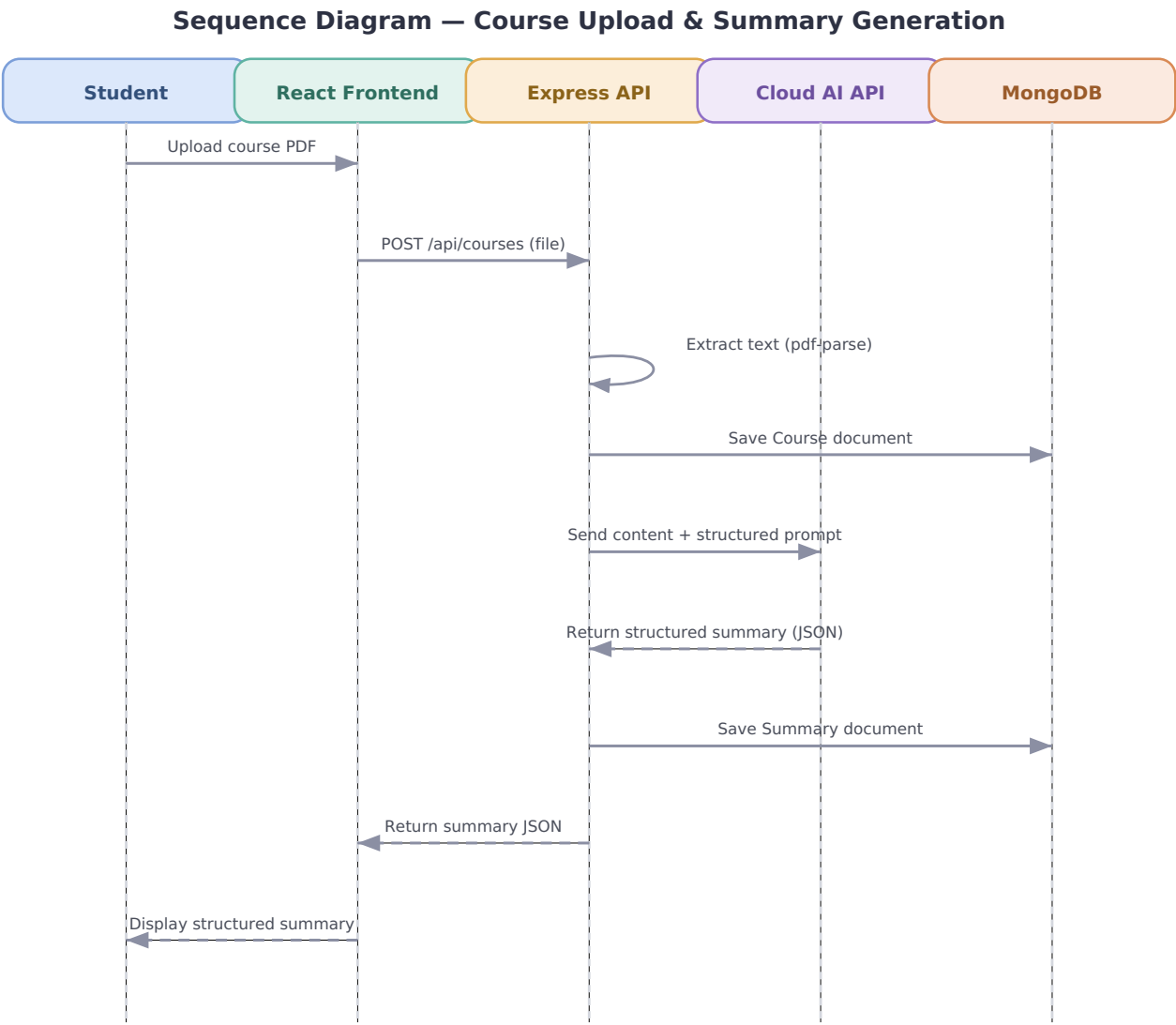
9 Class Diagram (Data Model)

This class diagram illustrates how the six main MongoDB collections relate to one another: a user owns several courses, and each course produces its own summaries, quizzes, and exam predictions, while progress links a user's performance back to a specific course.



10 Sequence Diagram (General Flow)

This sequence diagram traces the most representative flow in the application: a student uploads a course, the backend extracts and stores the text, the cloud AI API generates a structured summary, and the result is saved and displayed back to the student.

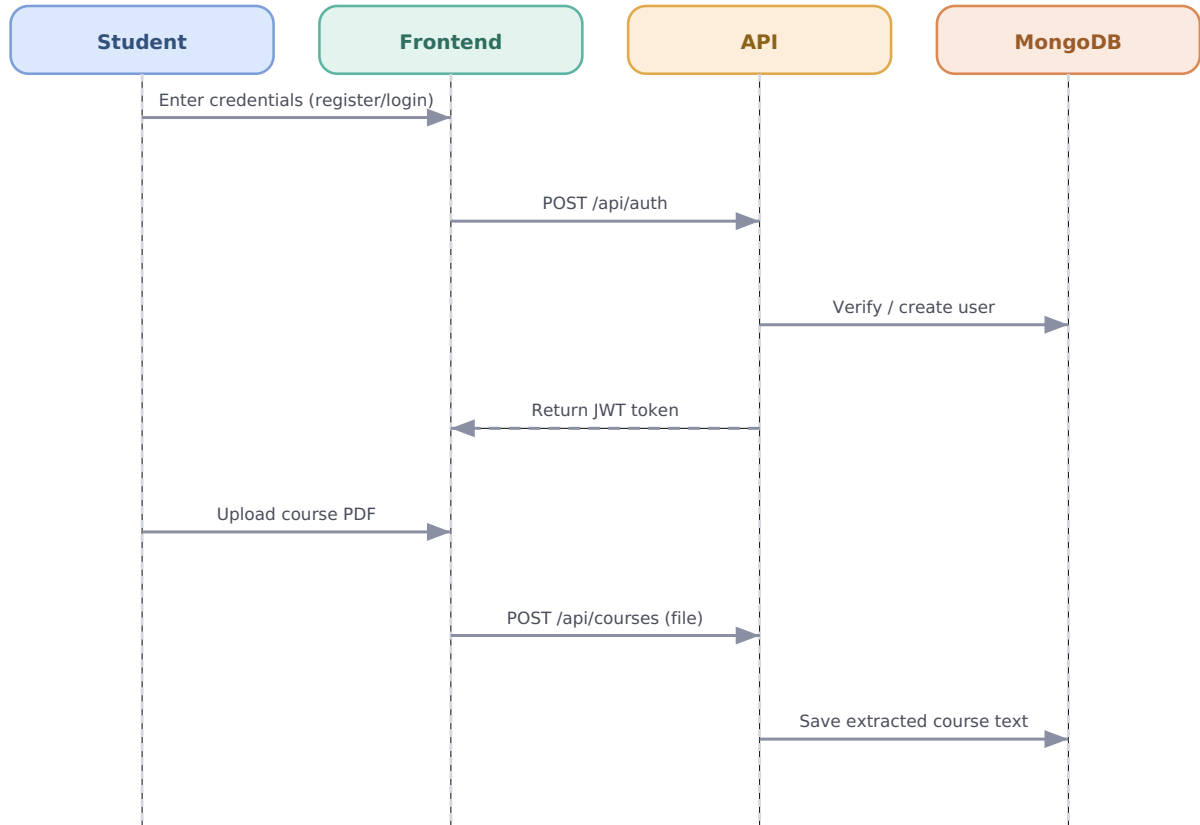


11 Sprint-by-Sprint Diagrams

Each sprint below is illustrated with its own sequence (or pipeline) diagram, showing exactly which interactions between the student, the frontend, the API, the AI service, and the database were implemented at that stage.

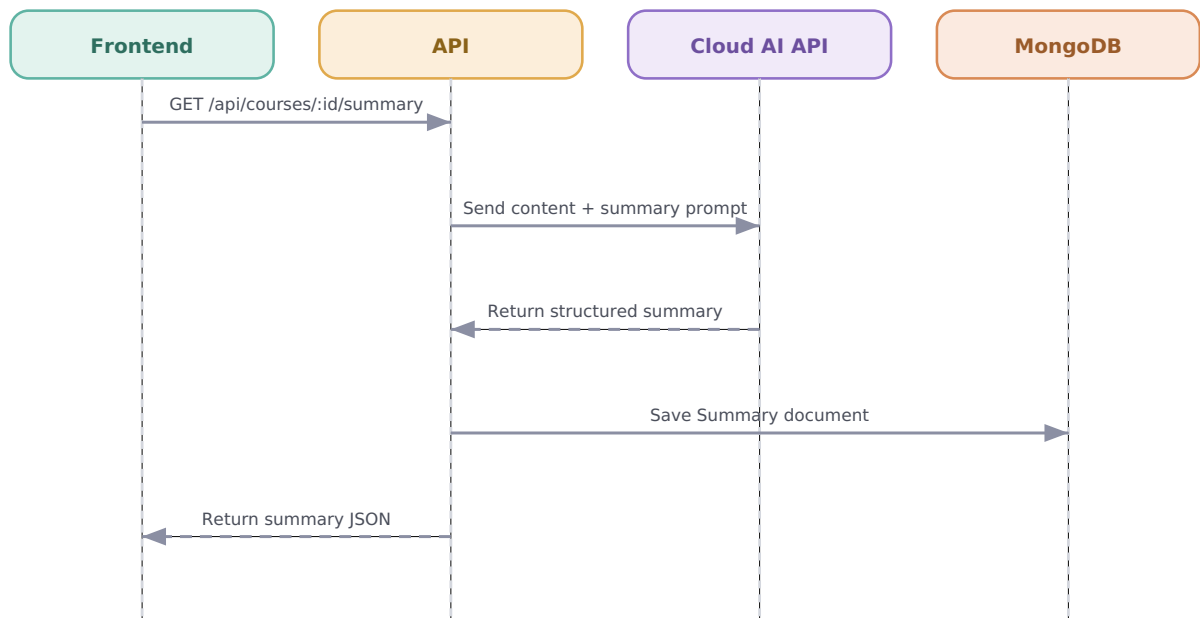
Sprint 1 — Setup, Authentication & Upload

Sprint 1 — Setup, Authentication & Upload



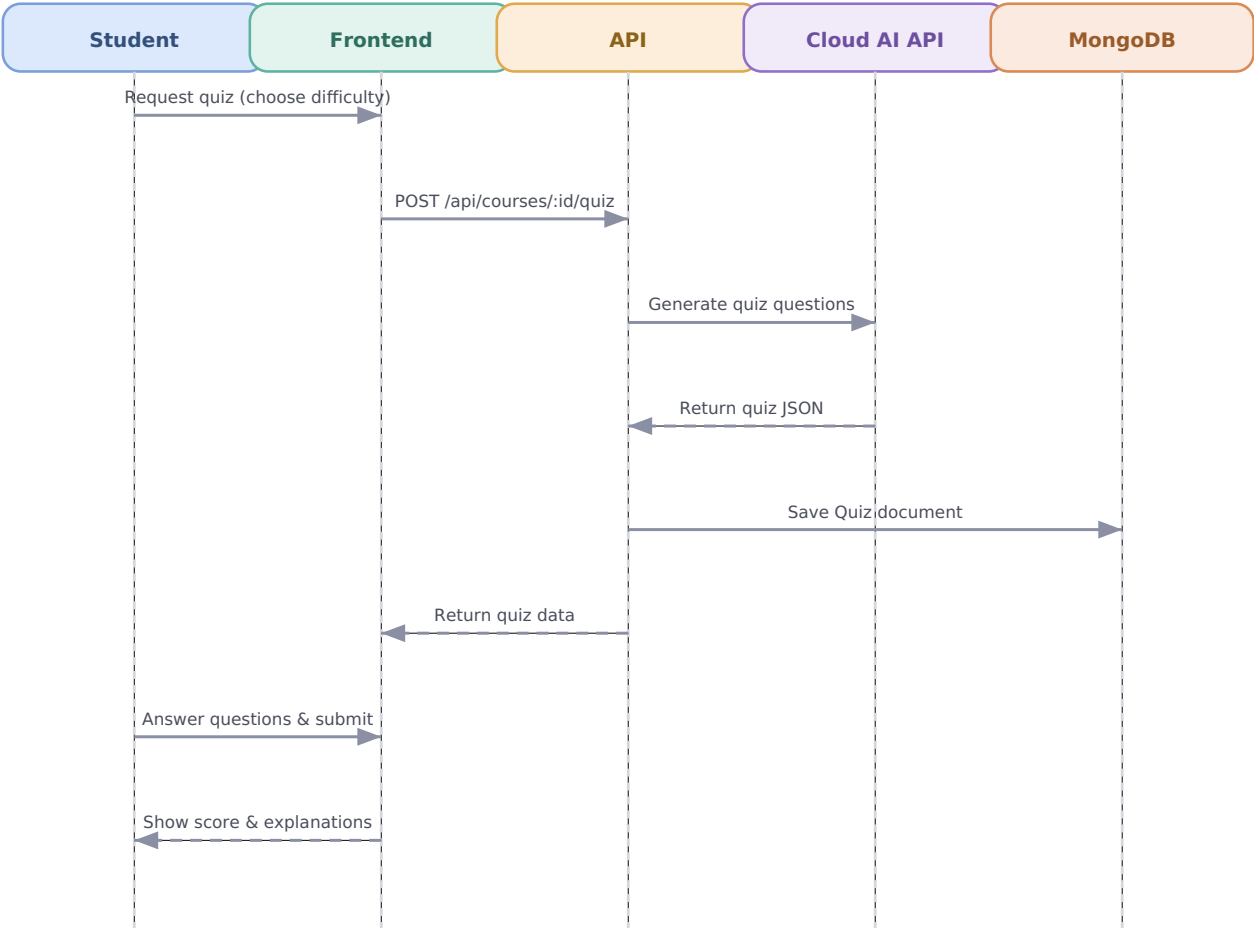
Sprint 2 — AI-Powered Summary Generation

Sprint 2 — AI-Powered Summary Generation



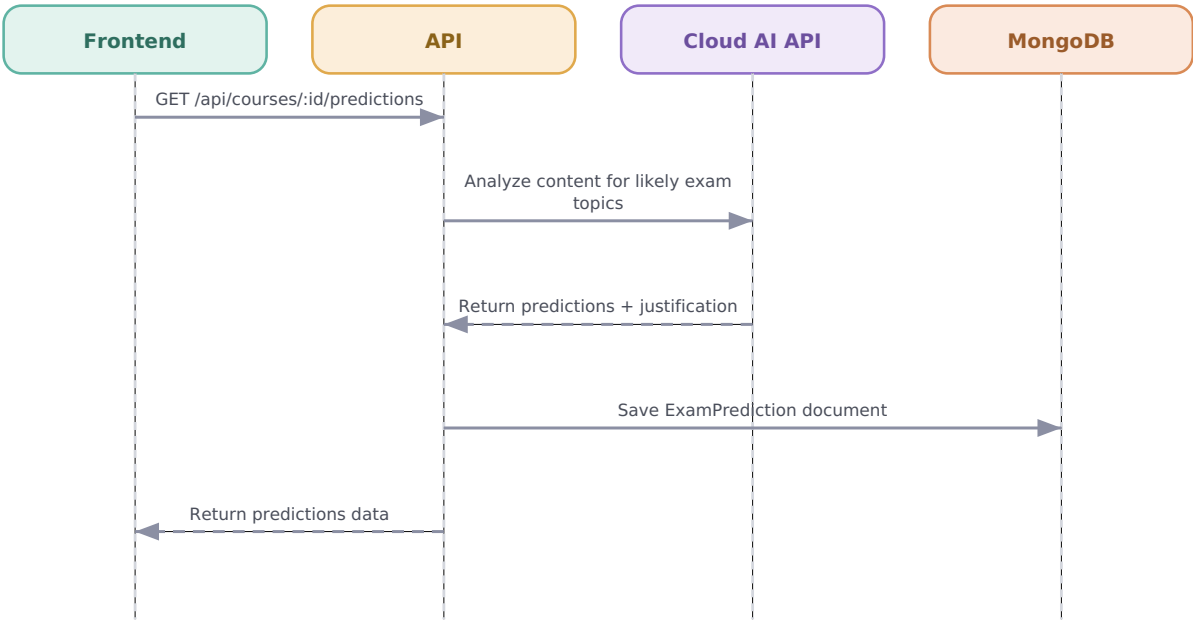
Sprint 3 — Quiz Generation & Quiz-Taking

Sprint 3 — Quiz Generation & Quiz-Taking



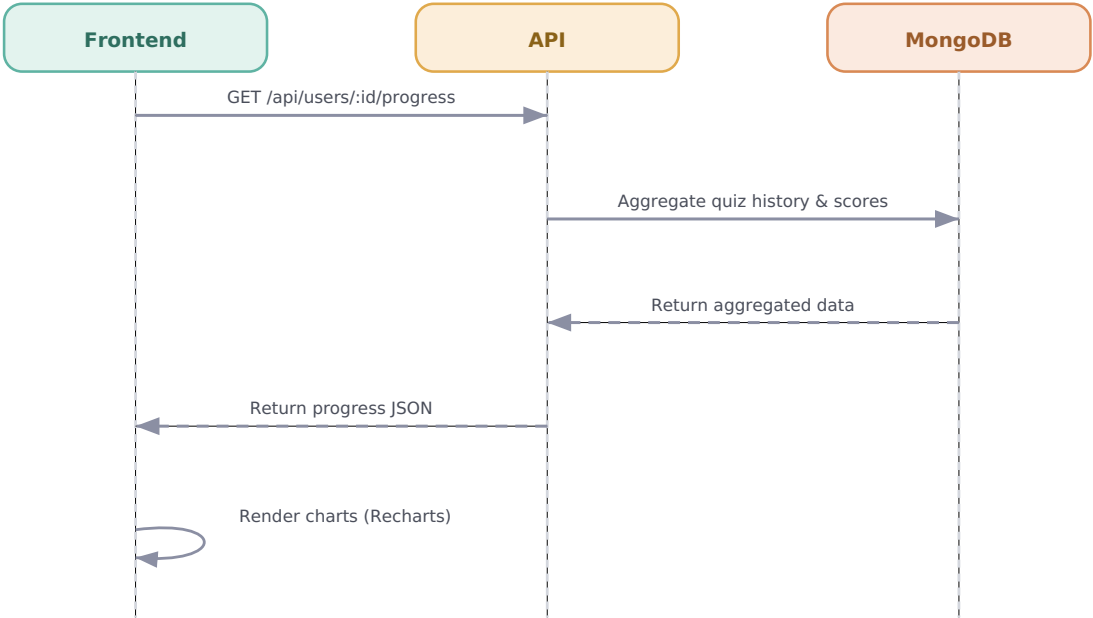
Sprint 4 — Exam Prediction Module

Sprint 4 — Exam Prediction Module



Sprint 5 — Progress Dashboard

Sprint 5 — Progress Dashboard



Sprint 6 — UI Polish, Testing & Deployment

Sprint 6 — UI Polish, Testing & Deployment

